

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO



Programas Clase 21

Unidad de Aprendizaje: *Sistemas Distribuidos*

Profesor: *Coronilla Contreras Ukranio*

Alumnos:

De la Rosa Gutiérrez Kaleb Yael

Grupo*:* *7CM2*

Ejercicio 1

```
import java.security.MessageDigest;
import java.nio.charset.StandardCharsets;

public class Ejercicio1 {

    public static void main(String[] args) throws Exception {

        String passwordSecreta = (args.length > 0) ? args[0] : "zorro";

        String hashObjetivo = sha256(passwordSecreta);


System.out.println("============================================================
=====");
        System.out.println(" Hash objetivo: " + hashObjetivo);
        System.out.println(" Buscando contraseña de 5 letras para: '" +
passwordSecreta + "'");

System.out.println("============================================================
=====");
        System.out.println("Iniciando busqueda exhaustiva por fuerza
bruta...");
        System.out.println("(Se mostrara el avance conforme se pruebe cada
letra inicial)\n");

        long inicio = System.currentTimeMillis();

        long intentos = 0;

        for (char a = 'a'; a <= 'z'; a++) {
            System.out.println("-> [" + a + "] Evaluando espacio '" + a +
"aaaa' hasta '" + a + "zzzz' (intentos acumulados: " + intentos + ")...");

            for (char b = 'a'; b <= 'z'; b++) {
                for (char c = 'a'; c <= 'z'; c++) {
                    for (char d = 'a'; d <= 'z'; d++) {
                        for (char e = 'a'; e <= 'z'; e++) {

                            String intento = "" + a + b + c + d + e;
```

```
                            intentos++;

                            String hash = sha256(intento);

                            if (hash.equalsIgnoreCase(hashObjetivo)) {

                                long fin = System.currentTimeMillis();


System.out.println("\n============================================================
=======");
                                System.out.println(" ¡CONTRASEÑA ENCONTRADA!:
" + intento);
                                System.out.println(" Intentos totales:       "
+ intentos);
                                System.out.println(" Tiempo transcurrido:    "
+ (fin - inicio) + " ms (" + String.format("%.2f", (fin - inicio) / 1000.0) +
" segundos)");

System.out.println("============================================================
=====");

                                return;
                            }
                        }
                    }
                }
            }
        }

        System.out.println("\nNo encontrada tras evaluar todo el espacio de
combinaciones.");
    }

    public static String sha256(String texto) throws Exception {

        MessageDigest md = MessageDigest.getInstance("SHA-256");

        byte[] hash = md.digest(texto.getBytes(StandardCharsets.UTF_8));

        StringBuilder hex = new StringBuilder();
```

```
        for (byte b : hash) {
            hex.append(String.format("%02x", b));
        }

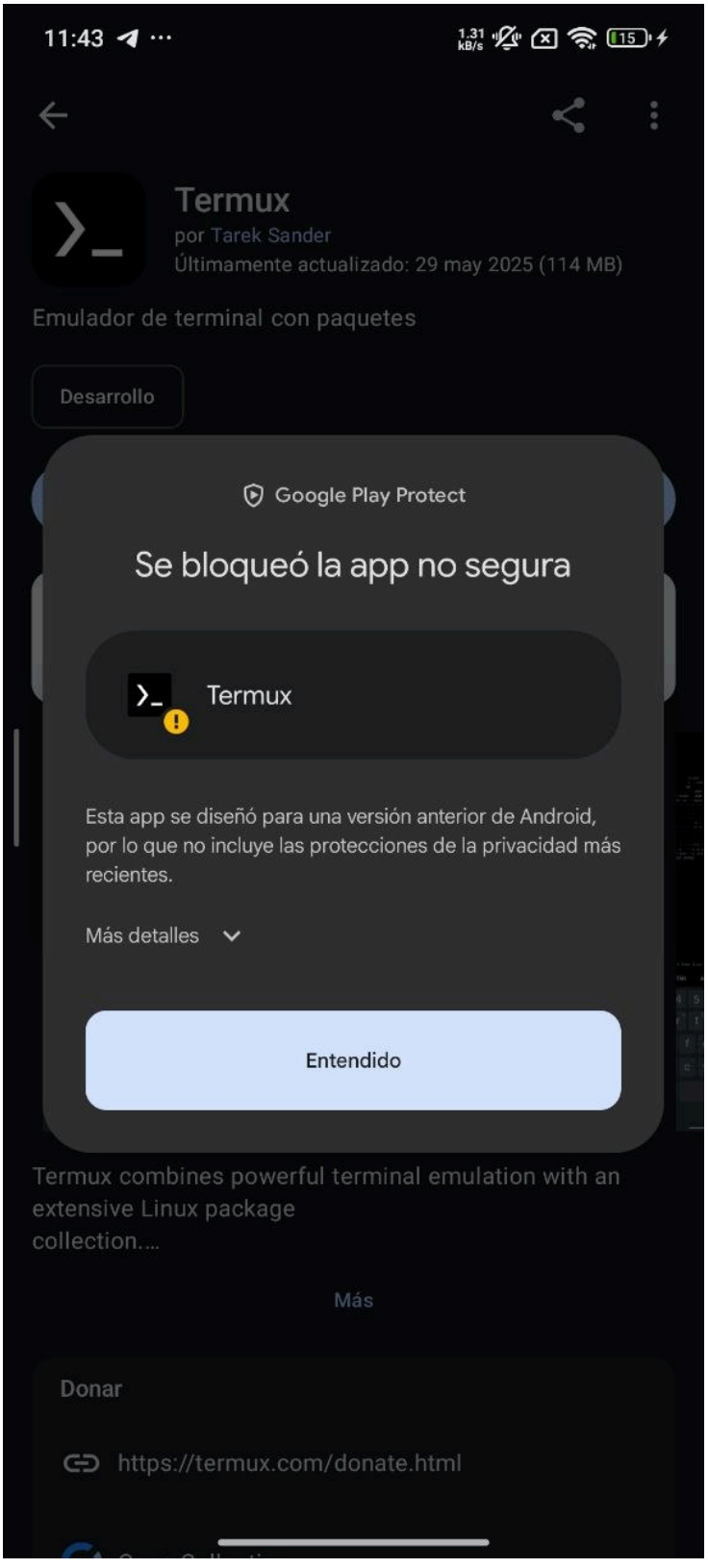
        return hex.toString();
    }
}
```

```
Administrador: Windows PowerShell
 Buscando contraseña de 5 letras para: 'zorro'
============================================================
Iniciando busqueda exhaustiva por fuerza bruta...
(Se mostrara el avance conforme se pruebe cada letra inicial)

-> [a] Evaluando espacio 'aaaaa' hasta 'azzzz' (intentos acumulados: 0)...
-> [b] Evaluando espacio 'baaaa' hasta 'bzzzz' (intentos acumulados: 456976)...
-> [c] Evaluando espacio 'caaaa' hasta 'czzzz' (intentos acumulados: 913952)...
-> [d] Evaluando espacio 'daaaa' hasta 'dzzzz' (intentos acumulados: 1370928)...
-> [e] Evaluando espacio 'eaaaa' hasta 'ezzzz' (intentos acumulados: 1827904)...
-> [f] Evaluando espacio 'faaaa' hasta 'fzzzz' (intentos acumulados: 2284880)...
-> [g] Evaluando espacio 'gaaaa' hasta 'gzzzz' (intentos acumulados: 2741856)...
-> [h] Evaluando espacio 'haaaa' hasta 'hzzzz' (intentos acumulados: 3198832)...
-> [i] Evaluando espacio 'iaaaa' hasta 'izzzz' (intentos acumulados: 3655808)...
-> [j] Evaluando espacio 'jaaaa' hasta 'jzzzz' (intentos acumulados: 4112784)...
-> [k] Evaluando espacio 'kaaaa' hasta 'kzzzz' (intentos acumulados: 4569760)...
-> [l] Evaluando espacio 'laaaa' hasta 'lzzzz' (intentos acumulados: 5026736)...
-> [m] Evaluando espacio 'maaaa' hasta 'mzzzz' (intentos acumulados: 5483712)...
-> [n] Evaluando espacio 'naaaa' hasta 'nzzzz' (intentos acumulados: 5940688)...
-> [o] Evaluando espacio 'oaaaa' hasta 'ozzzz' (intentos acumulados: 6397664)...
-> [p] Evaluando espacio 'paaaa' hasta 'pzzzz' (intentos acumulados: 6854640)...
-> [q] Evaluando espacio 'qaaaa' hasta 'qzzzz' (intentos acumulados: 7311616)...
-> [r] Evaluando espacio 'raaaa' hasta 'rzzzz' (intentos acumulados: 7768592)...
-> [s] Evaluando espacio 'saaaa' hasta 'szzzz' (intentos acumulados: 8225568)...
-> [t] Evaluando espacio 'taaaa' hasta 'tzzzz' (intentos acumulados: 8682544)...
-> [u] Evaluando espacio 'uaaaa' hasta 'uzzzz' (intentos acumulados: 9139520)...
-> [v] Evaluando espacio 'vaaaa' hasta 'vzzzz' (intentos acumulados: 9596496)...
-> [w] Evaluando espacio 'waaaa' hasta 'wzzzz' (intentos acumulados: 10053472)...
-> [x] Evaluando espacio 'xaaaa' hasta 'xzzzz' (intentos acumulados: 10510448)...
-> [y] Evaluando espacio 'yaaaa' hasta 'yzzzz' (intentos acumulados: 10967424)...
-> [z] Evaluando espacio 'zaaaa' hasta 'zzzzz' (intentos acumulados: 11424400)...

============================================================
 ¡CONTRASEÑA ENCONTRADA!: zorro
 Intentos totales:       11682413
 Tiempo transcurrido:    140021 ms (140.02 segundos)
============================================================
PS D:\SistemasDistribuidos\21>
```

Ejercicio 2
No logre instalar termux :c



Ejercicio 3

```java
import com.sun.net.httpserver.Headers;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;

import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.util.concurrent.Executors;

/**
 * Ejercicio 3: Servidor HTTP Concurrente para Búsqueda Distribuida de Hashes SHA-256.
 * Endpoint /task: recibe "<hashObjetivo>,<primeraLetra>" vía POST.
 * Examina el espacio de combinaciones desde <primeraLetra>aaaa hasta <primeraLetra>zzzz.
 * Devuelve "La contraseña es: <palabra>\n" o "NULL" si no la encuentra.
 * Cabecera X-debug-info: tiempo transcurrido en nanosegundos, segundos y milisegundos.
 */
public class ServidorHttp {

    private static final int PUERTO_DEFAULT = 8080;
    private static final int NUM_HILOS = 8;

    public static void main(String[] args) throws IOException {
        int puerto = PUERTO_DEFAULT;
        if (args.length > 0) {
            try {
                puerto = Integer.parseInt(args[0]);
            } catch (NumberFormatException e) {
                System.out.println("Puerto no válido. Usando puerto por defecto: " + PUERTO_DEFAULT);
                puerto = PUERTO_DEFAULT;
            }
        }
```

```
        HttpServer server = HttpServer.create(new InetSocketAddress(puerto),
0);
        server.setExecutor(Executors.newFixedThreadPool(NUM_HILOS));

        server.createContext("/task", new TaskHandler());
        server.createContext("/", new RaizHandler());

        server.start();
        System.out.println("Servidor HTTP (Ejercicio 3) iniciado en
http://localhost:" + puerto + "/task");
        System.out.println("Esperando solicitudes POST...");
    }

    static class TaskHandler implements HttpHandler {

        @Override
        public void handle(HttpExchange exchange) throws IOException {
            long nanoInicio = System.nanoTime();

            String metodo = exchange.getRequestMethod();
            Headers headers = exchange.getRequestHeaders();

            // Detectar cabecera X-Debug: true
            String xDebugValor = headers.getFirst("X-Debug");
            boolean esDebug = "true".equalsIgnoreCase(xDebugValor);

            if ("POST".equalsIgnoreCase(metodo)) {
                // Leer cuerpo de la petición
                InputStream is = exchange.getRequestBody();
                ByteArrayOutputStream buffer = new ByteArrayOutputStream();
                byte[] temp = new byte[1024];
                int leidos;
                while ((leidos = is.read(temp, 0, temp.length)) != -1) {
                    buffer.write(temp, 0, leidos);
                }
                String cuerpo = new String(buffer.toByteArray(),
StandardCharsets.UTF_8).trim();

                System.out.println("\n[POST /task] Petición recibida: <" +
cuerpo + ">");
```

```
                String respuestaCliente;
                String[] partes = cuerpo.split(",");

                if (partes.length < 2) {
                    respuestaCliente = "NULL";
                } else {
                    String hashObjetivo = partes[0].trim();
                    String letraStr = partes[1].trim();

                    if (letraStr.isEmpty()) {
                        respuestaCliente = "NULL";
                    } else {
                        char primeraLetra = letraStr.toLowerCase().charAt(0);
                        String encontrada = buscarClave(hashObjetivo,
primeraLetra);

                        if (encontrada != null) {
                            // Cuando encuentra la contraseña:
                            // Formato según especificación: "La contraseña
es: escom\n" (25 bytes)
                            respuestaCliente = "La contraseña es: " +
encontrada + "\n";
                            System.out.println(">> ¡Contraseña encontrada!: "
+ encontrada);
                        } else {
                            // Cuando NO encuentra la contraseña:
                            // Formato según especificación: "NULL" (4 bytes,
sin salto de línea)
                            respuestaCliente = "NULL";
                            System.out.println(">> Contraseña NO encontrada en
rango '" + primeraLetra + "aaaa' - '" + primeraLetra + "zzzz'. Retornando
NULL.");
                        }
                    }
                }

                // Si se solicitó X-Debug, calcular duración y adjuntar
cabecera
                if (esDebug) {
                    long nanoFin = System.nanoTime();
                    long duracionNanos = nanoFin - nanoInicio;
                    long segundos = duracionNanos / 1_000_000_000L;
```

```
                    long milisegundos = (duracionNanos % 1_000_000_000L) /
1_000_000L;

                    String debugInfo = String.format(
                            "La operacion tomo %d nanosegundos = %d segundos
con %d milisegundos",
                            duracionNanos, segundos, milisegundos
                    );
                    exchange.getResponseHeaders().set("X-debug-info",
debugInfo);
                    System.out.println(">> " + debugInfo);
                }

                byte[] bytesRespuesta =
respuestaCliente.getBytes(StandardCharsets.UTF_8);
                exchange.sendResponseHeaders(200, bytesRespuesta.length);
                OutputStream os = exchange.getResponseBody();
                os.write(bytesRespuesta);
                os.close();
                return;
            }

            // Si no es POST, devolver 405 Method Not Allowed
            String noPermitido = "Método no permitido. Utilice POST en
/task\n";
            byte[] bytesError = noPermitido.getBytes(StandardCharsets.UTF_8);
            exchange.sendResponseHeaders(405, bytesError.length);
            OutputStream os = exchange.getResponseBody();
            os.write(bytesError);
            os.close();
        }

        /**
         * Búsqueda por fuerza bruta de 4 caracteres para una primera letra
fija.
         * Rango: <primeraLetra>aaaa hasta <primeraLetra>zzzz (26^4 = 456,976
combinaciones).
         */
        private String buscarClave(String hashObjetivo, char primeraLetra) {
            char a = primeraLetra;

            try {
```

```
                for (char b = 'a'; b <= 'z'; b++) {
                    for (char c = 'a'; c <= 'z'; c++) {
                        for (char d = 'a'; d <= 'z'; d++) {
                            for (char e = 'a'; e <= 'z'; e++) {
                                String intento = "" + a + b + c + d + e;
                                String hash = sha256(intento);
                                if (hash.equalsIgnoreCase(hashObjetivo)) {
                                    return intento;
                                }
                            }
                        }
                    }
                }
            } catch (Exception ex) {
                System.err.println("Error en cálculo hash: " +
ex.getMessage());
            }

            return null;
        }

        /**
         * Cálculo de hash SHA-256 idéntico al del Ejercicio 1 / TestSHA
         */
        public static String sha256(String texto) throws Exception {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            byte[] hash = md.digest(texto.getBytes(StandardCharsets.UTF_8));
            StringBuilder hex = new StringBuilder();
            for (byte b : hash) {
                hex.append(String.format("%02x", b));
            }
            return hex.toString();
        }
    }

    static class RaizHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            String respuesta = "Servidor HTTP Ejercicio 21 activo. Envíe
peticiones POST a /task\n";
            byte[] bytes = respuesta.getBytes(StandardCharsets.UTF_8);
            exchange.sendResponseHeaders(200, bytes.length);
```

```
            OutputStream os = exchange.getResponseBody();
            os.write(bytes);
            os.close();
        }
    }
}
```

